

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch10_evaluation

AYuSh

Contents

Chapter 10: Model Evaluation & Metrics	3
--	---

Chapter 10: Model Evaluation & Metrics

A model is only as trustworthy as the number you judge it by, and most bad deployments trace back to a metric that lied — accuracy on imbalanced data, ROC-AUC where precision was what mattered, an offline R^2 inflated by leakage, or a "2% better" model that was pure sampling noise. Chapter 9 already leaned on this: it evaluated imbalance handling with PR-AUC precisely because accuracy is a broken compass at 3% prevalence. This chapter is where metric choice becomes a first-class skill. The organizing idea is that every metric encodes a value judgment — what counts as a mistake, how much a confident mistake costs, whether ranking or calibration or a point estimate is the deliverable — and choosing the metric *is* choosing what the model optimizes for once it reaches production.

The chapter walks the territory in order: the confusion matrix and the rates built from it (accuracy's failure staged at 6% prevalence, Listing 1); the ROC-vs-PR question, where ROC-AUC stays near 0.95 while PR-AUC falls toward the prevalence line as positives thin out (Listing 2); thresholds as a separate decision from the model, with the default 0.5 shown suboptimal and operating points chosen on validation data (Listing 3); calibration, where a margin-based classifier's probabilities are reshaped by Platt and isotonic and scored by Brier and log loss (Listing 4); proper scoring rules and why accuracy is blind to confidence (Listing 5); regression metrics and their characteristic failure modes — RMSE's outlier sensitivity, MAPE's explosion near zero, R^2 's negativity, adjusted R^2 's penalty for junk features (Listing 6); ranking metrics MRR/MAP/NDCG computed from scratch (Listing 7); the statistics of model comparison — McNemar's test and bootstrap confidence intervals that separate a real gain from noise (Listing 8); and conformal prediction, a distribution-free coverage guarantee (Listing 9). One rule threads through: report the metric that matches the decision, with an honesty interval around it.

The confusion matrix and its rates

Every classification metric for hard predictions is a ratio built from four counts: true positives (TP), false positives (FP), true negatives (TN), false negatives (FN). **Accuracy** = $(TP+TN)/all$ is the seductive default and the first casualty of imbalance — Listing 1 trains on 6% positives and scores 0.953 accuracy while a model that *always predicts negative* scores 0.938; the 1.5-point "skill" hides that recall is 0.367, i.e. the model misses nearly two-thirds of the positives it exists to catch. **Precision** = $TP/(TP+FP)$ answers "when it says positive, how often is it right?" — the metric you optimize when false alarms are costly (spam that eats real mail, a fraud freeze on a good customer). **Recall** (sensitivity, TPR) = $TP/(TP+FN)$ answers "of the real positives, how many did it catch?" — the metric you optimize when misses are costly (cancer screening, fraud that clears). The two trade

off through the threshold, and naming *which* error is more expensive is the first move in any evaluation answer.

F1 = $2 \cdot \text{precision} \cdot \text{recall} / (\text{precision} + \text{recall})$ is their harmonic mean — it punishes imbalance between the two (you cannot game it by maximizing one), which is why it beats accuracy on skewed classes; **F β** tilts the balance ($\beta > 1$ weights recall, $\beta < 1$ weights precision). **Balanced accuracy** = $(\text{TPR} + \text{TNR}) / 2$ averages per-class recall, so the always-negative baseline drops to 0.5 (Listing 1: 0.680, honestly mediocre). **Specificity** (TNR) = $\text{TN} / (\text{TN} + \text{FP})$ is recall for the negative class. The interview reflex: when you hear "imbalanced," stop quoting accuracy and switch to precision/recall/F1 or a class-balanced metric, and state the cost asymmetry that picks between them.

ROC-AUC versus PR-AUC

Both summarize a *ranking* of scores across all thresholds, but they answer different questions and disagree sharply under imbalance. The **ROC curve** plots TPR against FPR ($= \text{FP} / (\text{FP} + \text{TN})$); **ROC-AUC** is the probability a random positive outranks a random negative — a clean, prevalence-independent measure of discrimination, and 0.5 is chance. That prevalence-independence is exactly its blind spot: FPR has the giant true-negative pool in its denominator, so on rare positives a model can rack up many false positives while FPR barely moves. Listing 2 holds the model's discrimination roughly fixed and thins positives from 30% to 1.5%: ROC-AUC drifts only $0.951 \rightarrow 0.756$, while **PR-AUC** (average precision, the area under precision-vs-recall) collapses $0.867 \rightarrow 0.266$, tracking toward the no-skill line that for PR is the prevalence.

The rule: **use PR-AUC (or precision/recall at an operating point) when the positive class is rare and it's the class you care about** — fraud, disease, retrieval — because precision has FP against TP, not against the vast TN pool, so it feels every false alarm. Use **ROC-AUC when classes are roughly balanced or you genuinely care about both errors symmetrically**, or when you want a threshold-free discrimination number comparable across datasets of different prevalence. A subtle corollary: ROC-AUC is invariant to prevalence, so a model can look identically good on a balanced offline sample and a 1%-positive production stream while its *precision* — the thing the on-call team actually experiences — quietly craters. PR-AUC would have warned you.

Thresholds and operating points

A probabilistic classifier outputs scores; turning them into decisions needs a threshold, and **the default 0.5 is a convention, not an optimum**. The threshold is a business decision layered on top of the model: it slides you along the precision/recall curve, and where you sit should be set by the cost of each error, not by a library default. Listing 3 makes this concrete on 8% positives — the default 0.5 yields precision 0.974 / recall 0.664; dropping the threshold to the validation-chosen max-F1 point (0.360) trades precision down to 0.893 to buy recall up to 0.749 and a better F1 (0.815); demanding

precision ≥ 0.95 pushes the threshold back up to 0.510, buying precision at recall's expense. Same model, same scores — three different products.

Two disciplines separate candidates. First, **choose the threshold on a validation set, never on the test set** — the threshold is a fitted parameter, and tuning it on test is the same leakage sin as tuning a hyperparameter there (Listing 3 selects on a held-out validation split, then reports on test, which is why the realized test precision doesn't hit the target exactly — an honest reminder that operating points don't transfer perfectly). Second, for problems with an explicit cost matrix, **optimize expected cost directly** rather than F1: threshold where marginal cost of a false positive equals marginal benefit of a true positive. Threshold moving is also the cheapest fix for class imbalance (Chapter 9): train on the natural distribution, then move the threshold — often beating resampling, which mostly just relocates the implicit threshold anyway.

Probability calibration

A model is **calibrated** if, among all cases it scores 0.7, about 70% are actually positive — the scores mean what they say as probabilities. Discrimination (ranking, ROC-AUC) and calibration are independent: a model can rank perfectly yet be badly miscalibrated, and calibration is what you need whenever a downstream decision consumes the *probability itself* — expected-value thresholds, risk pricing, triage queues, or feeding scores into another model. Some learners are calibrated out of the box (logistic regression, by construction); many are not — SVMs output margins, naive Bayes' independence assumption makes it overconfident, and boosted trees push scores toward the extremes. Listing 4 shows an SVM's naive-sigmoid scores tracing a textbook miscalibration: cases it scores 0.15 are positive only 1% of the time, cases it scores 0.85 are positive 98% of the time — an S-shaped reliability curve that a raw Brier score of 0.061 quietly reflects.

The fixes are post-hoc, fit on a held-out calibration set. **Platt scaling** fits a one-parameter logistic (a sigmoid) mapping raw scores to probabilities — cheap, strong when the miscalibration is genuinely sigmoidal, and safe on small calibration sets. **Isotonic regression** fits a free monotonic step function — more flexible, corrects any monotone distortion, but needs more data and can overfit a small calibration set. Listing 4 cuts Brier $0.061 \rightarrow 0.043$ and log loss $0.249 \rightarrow 0.162$ with Platt, isotonic essentially tied here (more data would favor isotonic). Read the reliability curve to *diagnose*, then pick the scaler by data budget: **Platt when calibration data is scarce or the distortion looks sigmoidal; isotonic when you have plenty and the distortion is some arbitrary monotone shape**. Calibrate on data the model didn't train on — calibrating on the training set relearns the training distribution's optimism.

Proper scoring rules: log loss and Brier

A **proper scoring rule** is minimized (in expectation) by reporting your true probabilities — it rewards honesty, so you can't improve your expected score by shading predictions

away from your real belief. The two standard ones grade probabilities, not hard labels. **Log loss** (cross-entropy) = $-\text{mean}(\log p \text{ at the true class})$ punishes confident errors without mercy: a probability approaching 0 on an event that happens sends it toward infinity — Listing 5 clips a ~ 0 probability on a positive to a log loss of 34.5 from that single row. It's unbounded above, which makes it the sharpest instrument and the most sensitive to a single catastrophic overconfidence. **Brier score** = mean squared error between probability and outcome — bounded in $[0,1]$, gentler on outliers, and it decomposes into *reliability* – *resolution* + *uncertainty* (calibration, discrimination, and irreducible base-rate variance).

Listing 5 stages why these beat accuracy: a timid model that always assigns 0.6 to the correct class has perfect hard-label accuracy (1.000) but log loss 0.511, while a bold model that assigns 0.95 and is confidently wrong 8% of the time has *lower* accuracy (0.902) yet *lower* log loss (0.340) — because well-placed confidence, most of the time, pays off under a proper scoring rule even with occasional disasters. The lessons compound: accuracy is a step function blind to confidence, so two models with identical accuracy can have very different log loss; proper scoring rules are the right training and selection objective when you'll consume probabilities; and log loss vs Brier is a sensitivity choice — reach for log loss when a single overconfident miss is unacceptable, Brier when you want a bounded, decomposable, outlier-tolerant number.

Regression metrics

Regression metrics differ in what error they punish and where they break. **MSE** and its rooted, unit-restoring cousin **RMSE** square the residual, so they are dominated by large errors — Listing 6 injects three big misses and RMSE jumps $5.11 \rightarrow 6.75$ while **MAE** (mean absolute error) barely moves $4.08 \rightarrow 4.40$. That is the central trade: RMSE if large errors are disproportionately bad (and if you want the metric your squared-loss model actually optimizes); MAE if every unit of error is equally bad and you want robustness to outliers (MAE is minimized by the median, RMSE by the mean). **MAPE** (mean absolute percentage error) reports scale-free percentages — attractive for cross-series comparison, lethal near zero: Listing 6 sets five targets to 0.1 and MAPE explodes $8.5\% \rightarrow 497\%$ on unchanged predictions, and it asymmetrically punishes over-prediction. Prefer **sMAPE** or **WAPE** when targets approach zero.

$R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ is the fraction of variance explained, benchmarked against predicting the mean; it is unitless and comparable, but it **can go negative** — a model worse than the mean baseline (Listing 6: predicting 0 everywhere scores -24.6), a fact that surprises candidates who think R^2 lives in $[0,1]$. And R^2 *never decreases* when you add features, even pure noise, because in-sample fit can only improve — Listing 6 adds 40 noise columns and raw R^2 ticks up $0.915 \rightarrow 0.919$. **Adjusted R^2** penalizes for parameters, so it falls $0.914 \rightarrow 0.912$ on those junk features — which is why adjusted R^2 (or an out-of-sample R^2) is the honest one for model comparison across different feature counts. The umbrella rule: report an error metric in the target's units (RMSE or MAE)

alongside a normalized one (R^2), and choose squared vs absolute by how much you fear large errors.

Ranking metrics

When the deliverable is an *ordering* — search results, recommendations, retrieval for RAG (Chapter 24) — the metric must reward putting relevant items near the top, which classification and even AUC miss. **MRR** (mean reciprocal rank) scores $1/\text{rank}$ of the *first* relevant hit, averaged over queries — the right metric when the user wants one good answer (a known-item search); Listing 7 gives 1.000 to a list with a relevant item first and 0.333 to one where the first relevant is at rank 3. **MAP** (mean average precision) averages precision at every relevant position — it rewards packing *all* relevants high, not just the first (Listing 7: 0.917 vs 0.411 for the same relevant set reordered). **NDCG** (normalized discounted cumulative gain) is the most general: it supports *graded* relevance (0–3, not just yes/no) and discounts gains logarithmically by position, then normalizes by the ideal ordering so scores are comparable across queries — Listing 7 scores best-first at 1.000 and worst-first at 0.614 on identical graded relevances.

The unifying point Listing 7 makes explicit: a metric that ignores order — $\text{recall}@k$ with k equal to the list length — calls the good and bad rankings *identical*, because they contain the same relevant set. Ranking quality lives in the permutation, so the metric must be permutation-sensitive and position-discounted. Choose by the product: MRR for one-right-answer navigation, MAP for binary relevance where completeness matters, NDCG when relevance is graded or you need cross-query comparability. All three are reported @ k (NDCG@10, MAP@100) because users see a finite window.

Statistical significance of model comparison

Model B scores higher than A on the test set — is the gap real or sampling noise? Two tools answer honestly. **McNemar's test** is the paired test for classifiers on the *same* test set: build the 2×2 table of the rows where they disagree (A-wrong/B-right vs A-right/B-wrong) and test whether the split is lopsided — only the discordant pairs carry information, because rows both models get right or both get wrong say nothing about which is better. Listing 8 finds 184 rows where B rescues A against 54 the other way, $\chi^2 = 58.3$, $p < 0.0001$ — a real difference. **Bootstrap confidence intervals** generalize to any metric: resample the test set with replacement, recompute the metric gap thousands of times, and read the 2.5/97.5 percentiles — Listing 8's AUC gap for B–A is +0.047 with 95% CI [+0.039, +0.055], excluding zero.

The discipline this installs is the antidote to leaderboard theater. Listing 8's second comparison pits two gradient-boosting models differing only by random seed: the AUC gap is +0.0022 with 95% CI [−0.0004, +0.0048] — **spanning zero**, so the "improvement" is noise and shipping on it is superstition. The rules: pair your tests when the models see the same data (McNemar, paired bootstrap, 5×2 cv paired t-test) because

paired tests cancel the variance from *which* rows landed in test; a single train/test split gives one noisy estimate, so cross-validate and report a spread, not a point; and always translate a metric delta into "is the interval clear of zero?" before claiming a win. This is the same anti-optimism instinct as the leakage rules — measure the uncertainty, don't just quote the mean.

Conformal prediction and uncertainty

Point predictions and even calibrated probabilities still don't come with a *guarantee*; **conformal prediction** supplies one that is distribution-free and finite-sample. Split conformal is startlingly simple: hold out a calibration set, define a **nonconformity score** (for classification, $1 - \text{predicted probability of the true class}$), take the appropriate quantile of those calibration scores, and at test time emit the *set* of labels whose score falls under that quantile. The theorem: for a chosen error rate α , the prediction set contains the true label with probability at least $1 - \alpha$, assuming only that the data is exchangeable — no distributional form, no asymptotics. Listing 9 hits 0.904 empirical coverage at a 90% target and 0.951 at 95%, with the finite-sample $(n+1)(1-\alpha)/n$ quantile correction doing the guaranteeing.

The price of the guarantee is **adaptive set size**: where the model is confident the set holds one label; where it's unsure the set grows to two or more, and an empty set can appear when nothing is plausible enough — the set size *is* the honesty, converting vague confidence into a concrete "these are the possibilities at 95%." This is the practitioner's entry to uncertainty quantification, sitting alongside Bayesian posteriors, deep ensembles, and MC-dropout (Chapter 13) — but conformal is the one you can bolt onto *any* trained model for a coverage promise. Interview framing: calibration makes the probabilities honest on average; conformal makes a per-prediction coverage guarantee you can put in a contract, in exchange for sometimes admitting you don't know.

Code implementations

Listing 1 — Confusion matrix: why accuracy lies on imbalanced data

```
"""Listing 1: confusion matrix -- why accuracy lies, and precision/recall/F1 at the
default threshold."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score,
                             recall_score, f1_score, balanced_accuracy_score)

X, y = make_classification(n_samples=8000, n_features=20, n_informative=6,
                          weights=[0.94, 0.06], random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
clf = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
pred = clf.predict(Xte)                                     # default 0.5 threshold
```



```

tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
print(f"prevalence (test) : {yte.mean():.3f}")
print(f"confusion TN {tn} FP {fp} FN {fn} TP {tp}")
print(f"accuracy : {accuracy_score(yte, pred):.3f}")
print(f"always-negative acc : {1 - yte.mean():.3f} (the dumb baseline)")
print(f"precision : {precision_score(yte, pred):.3f}")
print(f"recall (sensitivity): {recall_score(yte, pred):.3f}")
print(f"F1 : {f1_score(yte, pred):.3f}")
print(f"balanced accuracy : {balanced_accuracy_score(yte, pred):.3f}")

```

```

prevalence (test) : 0.062
confusion TN 2233 FP 17 FN 95 TP 55
accuracy : 0.953
always-negative acc : 0.938 (the dumb baseline)
precision : 0.764
recall (sensitivity): 0.367
F1 : 0.495
balanced accuracy : 0.680

```

Listing 2 — ROC-AUC vs PR-AUC as positives get rarer

```

"""Listing 2: ROC-AUC vs PR-AUC -- why ROC flatters a model on rare positives."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, average_precision_score

def evaluate(weight_pos, seed=0):
    X, y = make_classification(n_samples=20000, n_features=20, n_informative=6,
                              weights=[1 - weight_pos, weight_pos], random_state=seed)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, random_state=seed,
                                          stratify=y)
    s = LogisticRegression(max_iter=2000).fit(Xtr, ytr).predict_proba(Xte)[ :, 1]
    return yte.mean(), roc_auc_score(yte, s), average_precision_score(yte, s)

print(f"{'prevalence':>10} {'ROC-AUC':>9} {'PR-AUC':>8} {'no-skill PR (=prev)':>20}")
for w in (0.30, 0.10, 0.03, 0.01):
    prev, roc, pr = evaluate(w)
    print(f"{'prev':>10.3f} {'roc':>9.3f} {'pr':>8.3f} {'prev':>20.3f}")
print("\nROC-AUC barely moves as positives get rarer; PR-AUC falls toward the")
print("no-skill line (=prevalence). Same ranking model, same discrimination --")
print("ROC hides that precision is collapsing under a flood of true negatives.")

```

prevalence	ROC-AUC	PR-AUC	no-skill PR (=prev)
0.302	0.951	0.867	0.302
0.104	0.941	0.688	0.104
0.035	0.875	0.417	0.035
0.015	0.756	0.266	0.015

ROC-AUC barely moves as positives get rarer; PR-AUC falls toward the no-skill line (=prevalence). Same ranking model, same discrimination -- ROC hides that precision is collapsing under a flood of true negatives.

Listing 3 — Threshold moving: the default 0.5 is not the operating point

```

"""Listing 3: threshold moving -- the default 0.5 is almost never the right operating point."""

```

```

import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import precision_recall_curve, f1_score, precision_score,
recall_score

X, y = make_classification(n_samples=12000, n_features=20, n_informative=6,
                           weights=[0.92, 0.08], random_state=1)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, random_state=1, stratify=y)
# split a validation set to CHOOSE the threshold (never on test)
Xtr2, Xval, ytr2, yval = train_test_split(Xtr, ytr, test_size=0.33, random_state=1,
stratify=ytr)
m = RandomForestClassifier(300, random_state=1, n_jobs=-1).fit(Xtr2, ytr2)
sval = m.predict_proba(Xval)[: , 1]
ste = m.predict_proba(Xte)[: , 1]

def report(t, tag):
    p = (ste >= t).astype(int)
    print(f"{tag:22} thr {t:.3f} P {precision_score(yte,p):.3f} R
{recall_score(yte,p):.3f} F1 {f1_score(yte,p):.3f}")

report(0.5, "default 0.5")
# max-F1 threshold chosen on validation
prec, rec, thr = precision_recall_curve(yval, sval)
f1s = 2*prec*rec/(prec+rec+1e-12)
t_f1 = thr[np.nanargmax(f1s[:-1])]
report(t_f1, "max-F1 (val-chosen)")
# threshold for precision >= 0.95 on validation
ok = np.where(prec[:-1] >= 0.95)[0]
t_p90 = thr[ok[0]] if len(ok) else 0.99
report(t_p90, "precision>=0.95 target")

```

default 0.5	thr 0.500	P 0.974	R 0.664	F1 0.790
max-F1 (val-chosen)	thr 0.360	P 0.893	R 0.749	F1 0.815
precision>=0.95 target	thr 0.510	P 0.974	R 0.659	F1 0.786

Listing 4 — Calibration: reliability, Brier & log loss under Platt and isotonic

```

"""Listing 4: calibration -- reliability, Brier & log loss before/after Platt and
isotonic."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.model_selection import train_test_split
from sklearn.metrics import brier_score_loss, log_loss

X, y = make_classification(n_samples=15000, n_features=20, n_informative=8,
random_state=2)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, random_state=2, stratify=y)

base = SVC(probability=False).fit(Xtr, ytr) # margins -> sigmoid = badly
calibrated
# use decision function pushed through a naive sigmoid as the "raw" probability
raw = 1/(1+np.exp(-base.decision_function(Xte)))
platt = CalibratedClassifierCV(SVC(), method="sigmoid", cv=3).fit(Xtr,
ytr).predict_proba(Xte)[: , 1]
iso = CalibratedClassifierCV(SVC(), method="isotonic", cv=3).fit(Xtr,
ytr).predict_proba(Xte)[: , 1]

```

```

for tag, p in [("raw (naive sigmoid)", raw), ("Platt (sigmoid)", platt), ("isotonic",
iso)]:
    print(f"{tag:22} Brier {brier_score_loss(yte,p):.4f}    log-loss {log_loss(yte,p):.4f}")

# reliability: fraction positive vs mean predicted, 10 bins, for the raw model
frac, mean = calibration_curve(yte, raw, n_bins=10)
print("\nreliability (raw): predicted vs actual by bin")
for mp, fp in zip(mean, frac):
    print(f"  pred {mp:.2f}  ->  actual {fp:.2f}")

```

```

raw (naive sigmoid)    Brier 0.0607    log-loss 0.2491
Platt (sigmoid)       Brier 0.0428    log-loss 0.1621
isotonic              Brier 0.0431    log-loss 0.1675

```

```

reliability (raw): predicted vs actual by bin
  pred 0.07 -> actual 0.01
  pred 0.15 -> actual 0.01
  pred 0.24 -> actual 0.06
  pred 0.34 -> actual 0.12
  pred 0.45 -> actual 0.37
  pred 0.55 -> actual 0.64
  pred 0.65 -> actual 0.81
  pred 0.76 -> actual 0.95
  pred 0.85 -> actual 0.98
  pred 0.93 -> actual 0.99

```

Listing 5 — Proper scoring rules punish confident-wrong; accuracy can't

```

"""Listing 5: proper scoring rules -- log loss and Brier punish confident-wrong;
accuracy can't."""
import numpy as np
from sklearn.metrics import log_loss, brier_score_loss, accuracy_score

rng = np.random.default_rng(3)
n = 2000
y = rng.integers(0, 2, n)
# Model A: hesitant but honest -- correct class gets prob 0.6
# Model B: bold -- correct class gets prob 0.95 ... but 8% of the time it is boldly
WRONG at 0.99
pa = np.where(y == 1, 0.6, 0.4)
pb = np.where(y == 1, 0.95, 0.05).astype(float)
flip = rng.random(n) < 0.08
pb[flip] = 1 - pb[flip]                                # confident and wrong

for tag, p in [("A: hesitant (0.6)", pa), ("B: bold + 8% confident-wrong", pb)]:
    hard = (p >= 0.5).astype(int)
    print(f"{tag:32} acc {accuracy_score(y,hard):.3f}  logloss {log_loss(y,p):.3f}
Brier {brier_score_loss(y,p):.3f}")

# the single-prediction catastrophe: one prob of 0 on a positive -> infinite surprise
(clipped)
p_one = np.array([1e-15]); y_one = np.array([1])
print(f"\none confident-wrong prob (~0 on a positive): log loss {log_loss(y_one,
p_one, labels=[0,1]):.1f}")
print("Accuracy is a step function -- blind to confidence. Log loss is unbounded")
print("above and rewards honesty; Brier is bounded [0,1] and decomposes into")
print("reliability - resolution + uncertainty.")

```

```

A: hesitant (0.6)          acc 1.000  logloss 0.511  Brier 0.160
B: bold + 8% confident-wrong  acc 0.902  logloss 0.340  Brier 0.091

```

one confident-wrong prob (~0 on a positive): log loss 34.5
 Accuracy is a step function -- blind to confidence. Log loss is unbounded above and rewards honesty; Brier is bounded [0,1] and decomposes into reliability - resolution + uncertainty.

Listing 6 — Regression metrics: outliers, near-zero targets, junk features

```
"""Listing 6: regression metrics -- outliers, scale, near-zero targets, and junk
features."""
import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

rng = np.random.default_rng(4)
n = 500
y = rng.normal(50, 10, n)
pred = y + rng.normal(0, 5, n) # honest predictor, RMSE ~5
def row(tag, yt, yp):
    rmse = mean_squared_error(yt, yp) ** 0.5
    mae = mean_absolute_error(yt, yp)
    mape = np.mean(np.abs((yt - yp) / yt)) * 100
    print(f"{tag:20} RMSE {rmse:6.2f} MAE {mae:6.2f} MAPE {mape:6.1f}% R2
{r2_score(yt,yp):.3f}")

row("clean", y, pred)
# inject 3 large errors -> RMSE explodes, MAE barely moves (squared vs absolute)
pred2 = pred.copy(); pred2[:3] += 60
row("3 big errors", y, pred2)
# near-zero targets wreck MAPE
y0 = y.copy(); y0[:5] = 0.1
row("targets near 0", y0, pred) # same preds, tiny denominators

# R2 can go NEGATIVE: predicting a constant worse than the mean baseline
print(f"\nR2 of 'predict 0 for everything' : {r2_score(y, np.zeros(n)):.3f} (worse
than mean => negative)")

# adjusted R2 penalizes useless features
from sklearn.linear_model import LinearRegression
Xr = rng.normal(size=(n, 1)); ytrue = 3*Xr[:,0] + rng.normal(0, 1, n)
Xjunk = np.c_[Xr, rng.normal(size=(n, 40))] # 40 pure-noise columns
def adj(r2, n, p): return 1 - (1 - r2)*(n - 1)/(n - p - 1)
for name, XX in [("1 real feature", Xr), ("40 noise feats", Xjunk)]:
    r2 = LinearRegression().fit(XX, ytrue).score(XX, ytrue)
    print(f"{name:16} R2 {r2:.3f} adjR2 {adj(r2, n, XX.shape[1]):.3f}")
```

clean	RMSE	5.11	MAE	4.08	MAPE	8.5%	R2	0.743
3 big errors	RMSE	6.75	MAE	4.40	MAPE	9.1%	R2	0.552
targets near 0	RMSE	7.14	MAE	4.53	MAPE	497.4%	R2	0.593

R2 of 'predict 0 for everything' : -24.609 (worse than mean => negative)
 1 real feature R2 0.915 adjR2 0.914
 +40 noise feats R2 0.919 adjR2 0.912

Listing 7 — Ranking metrics from scratch: MRR, MAP, NDCG

```
"""Listing 7: ranking metrics from scratch -- MRR, MAP, NDCG reward the TOP of the
list."""
import numpy as np
```

```

def dcg(rels):
    rels = np.asarray(rels, float)
    return np.sum(rels / np.log2(np.arange(2, len(rels) + 2)))
def ndcg(rels):
    ideal = dcg(sorted(rels, reverse=True))
    return dcg(rels) / ideal if ideal > 0 else 0.0
def average_precision(binary):
    binary = np.asarray(binary)
    hits = np.cumsum(binary)
    precs = hits / (np.arange(len(binary)) + 1)
    return (precs * binary).sum() / binary.sum() if binary.sum() else 0.0
def rr(binary):
    idx = np.where(np.asarray(binary) == 1)[0]
    return 1.0 / (idx[0] + 1) if len(idx) else 0.0

# two rankings of the same 6 docs; relevance in {0,1} (graded for NDCG uses same)
good = [1, 1, 0, 1, 0, 0]      # relevants near the top
bad  = [0, 0, 1, 0, 1, 1]      # same # relevant, buried
print(f"{'ranking':8} {'MRR':>6} {'AP':>6} {'NDCG':>6}")
for tag, r in [("good", good), ("bad", bad)]:
    print(f"{'tag':8} {'rr(r)':6.3f} {'average_precision(r)':6.3f} {'ndcg(r)':6.3f}")

# graded relevance (0..3): NDCG rewards putting the MOST relevant first
graded_a = [3, 2, 1, 0]
graded_b = [0, 1, 2, 3]
print(f"\ngraded NDCG best-first {ndcg(graded_a):.3f} worst-first {ndcg(graded_b):.3f}")
print("Same relevant set, different order -> different score. A metric that ignores")
print("order (e.g. recall@k with k=6) would call good and bad identical.")

```

ranking	MRR	AP	NDCG
good	1.000	0.917	0.967
bad	0.333	0.411	0.583

```

graded NDCG best-first 1.000 worst-first 0.614
Same relevant set, different order -> different score. A metric that ignores
order (e.g. recall@k with k=6) would call good and bad identical.

```

Listing 8 — Is B really better? McNemar's test + bootstrap CI

```

"""Listing 8: is model B really better? McNemar's test + bootstrap CI on the metric gap."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score
from statsmodels.stats.contingency_tables import mcnemar

X, y = make_classification(n_samples=6000, n_features=25, n_informative=8,
                           random_state=5)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, random_state=5, stratify=y)
A = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
B = GradientBoostingClassifier(random_state=5, subsample=0.7).fit(Xtr, ytr)
pa, pb = A.predict(Xte), B.predict(Xte)
sa, sb = A.predict_proba(Xte)[:,1], B.predict_proba(Xte)[:,1]
print(f"accuracy A {accuracy_score(yte,pa):.4f} B {accuracy_score(yte,pb):.4f}")
print(f"ROC-AUC A {roc_auc_score(yte,sa):.4f} B {roc_auc_score(yte,sb):.4f}")

# McNemar: only the DISAGREEMENTS count (paired predictions on the same test rows)
b_only = int(((pa != yte) & (pb == yte)).sum()) # A wrong, B right
a_only = int(((pa == yte) & (pb != yte)).sum()) # A right, B wrong
res = mcnemar([[0, a_only], [b_only, 0]], exact=False, correction=True)

```

```

print(f"\nMcNemar discordant: A-wrong/B-right {b_only}, A-right/B-wrong {a_only}")
print(f"McNemar chi2 {res.statistic:.3f} p-value {res.pvalue:.4f}")

# bootstrap 95% CI on the AUC difference (B - A) over resampled test rows
rng = np.random.default_rng(5); diffs = []
for _ in range(2000):
    idx = rng.integers(0, len(yte), len(yte))
    if yte[idx].sum() in (0, len(idx)): continue
    diffs.append(roc_auc_score(yte[idx], sb[idx]) - roc_auc_score(yte[idx], sa[idx]))
lo, hi = np.percentile(diffs, [2.5, 97.5])
print(f"bootstrap AUC gap (B-A): {np.mean(diffs):+.4f} 95% CI [{lo:+.4f}, {hi:+.4f}]")
print("CI excludes 0 => the gap is real, not sampling noise." if lo > 0 else
      "CI includes 0 => cannot claim B beats A.")

# --- contrast: two GBMs differing only by random_state -> tiny gap, within noise ---
B2 = GradientBoostingClassifier(random_state=17, subsample=0.7).fit(Xtr, ytr)
sb2 = B2.predict_proba(Xte)[: ,1]
diffs2 = []
for _ in range(2000):
    idx = rng.integers(0, len(yte), len(yte))
    if yte[idx].sum() in (0, len(idx)): continue
    diffs2.append(roc_auc_score(yte[idx], sb2[idx]) - roc_auc_score(yte[idx], sb[idx]))
lo2, hi2 = np.percentile(diffs2, [2.5, 97.5])
print(f"\ntwo GBMs (seed 17 vs 5): AUC gap {np.mean(diffs2):+.4f} 95% CI [{lo2:+.4f}, {hi2:+.4f}]")
print("CI spans 0 => the seed-to-seed difference is noise; don't ship on it." if lo2
      <= 0 <= hi2
      else "CI excludes 0.")

```

```

accuracy  A 0.8400  B 0.8904
ROC-AUC   A 0.9116  B 0.9585

```

```

McNemar discordant: A-wrong/B-right 184, A-right/B-wrong 63
McNemar chi2 58.300 p-value 0.0000
bootstrap AUC gap (B-A): +0.0470 95% CI [+0.0393, +0.0554]
CI excludes 0 => the gap is real, not sampling noise.

```

```

two GBMs (seed 17 vs 5): AUC gap +0.0022 95% CI [-0.0004, +0.0048]
CI spans 0 => the seed-to-seed difference is noise; don't ship on it.

```

Listing 9 — Split conformal prediction: distribution-free coverage

```

"""Listing 9: split conformal prediction -- distribution-free coverage you can actually
trust."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=12000, n_features=20, n_informative=8,
                          n_classes=4, n_clusters_per_class=1, random_state=6)
Xtr, Xrest, ytr, yrest = train_test_split(X, y, test_size=0.5, random_state=6,
                                          stratify=y)
Xcal, Xte, ycal, yte = train_test_split(Xrest, yrest, test_size=0.5, random_state=6,
                                          stratify=yrest)
clf = RandomForestClassifier(300, random_state=6, n_jobs=-1).fit(Xtr, ytr)

# nonconformity = 1 - predicted prob of the TRUE class, scored on calibration set
cal_scores = 1 - clf.predict_proba(Xcal)[np.arange(len(ycal)), ycal]
for alpha in (0.10, 0.05):
    # finite-sample quantile (the (n+1)(1-alpha)/n correction gives the guarantee)
    n = len(cal_scores)
    qhat = np.quantile(cal_scores, np.ceil((n + 1) * (1 - alpha)) / n, method="higher")
    proba = clf.predict_proba(Xte)

```



```

sets = proba >= (1 - qhat)                                # include every class plausible
enough
covered = sets[np.arange(len(yte)), yte].mean()
avg_size = sets.sum(axis=1).mean()
print(f"target coverage {1-alpha:.0%} -> empirical {covered:.3f}    avg set size
{avg_size:.2f}")
print("\nGuarantee holds WITHOUT any distributional assumption: the prediction set")
print("contains the true label >= 1-alpha of the time, in exchange for sometimes")
print("returning more than one class (set size grows where the model is unsure).")

```

```

target coverage 90% -> empirical 0.904    avg set size 0.94
target coverage 95% -> empirical 0.951    avg set size 1.02

```

Guarantee holds WITHOUT any distributional assumption: the prediction set contains the true label $\geq 1 - \alpha$ of the time, in exchange for sometimes returning more than one class (set size grows where the model is unsure).

Pitfalls, comparisons and practical tips

Accuracy on imbalanced data. The evergreen trap: 95% accuracy at 5% prevalence can be worse than the always-negative baseline on the class you care about (Listing 1). The instant an interviewer says "imbalanced," drop accuracy for precision/recall/F1, PR-AUC, or balanced accuracy, and name the cost asymmetry.

ROC-AUC where PR-AUC belongs. ROC-AUC is prevalence-invariant, which is a feature for cross-dataset comparison and a bug for rare-positive problems — it stays high while precision collapses (Listing 2). Rare, valuable positive class $\rightarrow$ PR-AUC and precision/recall at an operating point.

Tuning the threshold on the test set. The decision threshold is a fitted parameter; choose it on validation, report on test (Listing 3). Same rule for choosing among calibration methods and for early-stopping rounds — anything selected by looking at a score is a parameter that can leak.

Confusing discrimination with calibration. A high AUC says the ranking is good, not that the probabilities are trustworthy. If a downstream decision reads the probability (expected value, pricing, triage), check a reliability curve and calibrate (Platt/isotonic) on held-out data (Listing 4). Never calibrate on the training set.

Optimizing accuracy when you'll consume probabilities. Accuracy is a step function blind to confidence; two models with equal accuracy can differ wildly in log loss (Listing 5). Train and select on a proper scoring rule (log loss, Brier) when the output is a probability, and remember log loss is unbounded — one confident-wrong prediction dominates.

MAPE near zero, RMSE with outliers, R^2 assumed bounded. MAPE explodes as targets approach zero (Listing 6) — use sMAPE/WAPE. RMSE is outlier-dominated; report MAE alongside if large errors are rare noise. R^2 can be negative (worse than the mean)

and never falls when features are added — use adjusted or out-of-sample R^2 for model comparison.

Order-blind metrics for ranking problems. Recall@k with k = list length ignores the permutation and calls a top-loaded and a bottom-loaded ranking identical (Listing 7). Use MRR/MAP/NDCG, reported @k, and pick by whether you need one hit, all hits high, or graded relevance.

Claiming a win inside the noise band. A metric delta is an estimate with variance. Pair the test (McNemar, paired bootstrap) and check whether the confidence interval clears zero before shipping — a seed-to-seed gap of 0.002 with a CI spanning zero is not an improvement (Listing 8). Cross-validate and report a spread, not a single split's point estimate.

Data leakage inflating every offline number. The whole chapter's metrics are honest only if the evaluation split is honest. All of Chapter 9's leakage traps (target encoding, SMOTE-before-split, selection-before-CV) manufacture beautiful offline scores that vanish in production — audit the pipeline before trusting the metric.

Metric-decision mismatch. The meta-pitfall: optimizing a metric that isn't the business objective. Precision/recall balance, threshold, and even the choice of ROC vs PR encode a cost model — write the cost model down first, then choose the metric that matches it.

Interview questions and answers

Q1. Define precision, recall, and specificity from the confusion matrix, and give a problem where each is the metric to maximize.

Precision = $TP/(TP+FP)$: of the flagged positives, how many are real — maximize when false alarms are expensive (spam filtering that must not eat real mail, a fraud freeze on good customers). Recall/sensitivity = $TP/(TP+FN)$: of the real positives, how many caught — maximize when misses are expensive (cancer screening, security). Specificity = $TN/(TN+FP)$: recall for the negative class, i.e. how well you clear the truly-negative. The interview move is naming the cost asymmetry first, because it's what picks the metric — precision and recall trade off through the threshold, so you can't max both without moving to F1 or fixing an operating point.

Q2. Why is accuracy misleading on imbalanced data, and what do you report instead?

Because the majority class dominates the count: at 6% positives a model that never fires scores 94% (Listing 1's baseline 0.938), so a "95%" model can be catching almost nothing — its recall was 0.367. Accuracy weights every row equally, which is exactly wrong when the rare class is the point. Report precision/recall/F1, PR-AUC, or balanced accuracy (per-class recall averaged, which drops the trivial baseline to 0.5), and state whether misses or false alarms cost more so the reader knows which of precision/recall to weight.

Q3. What is F1, why the harmonic mean, and when is it the wrong summary?

$F1 = 2PR/(P+R)$, the harmonic mean of precision and recall. The harmonic mean is dominated by the smaller of the two, so you can't inflate F1 by maxing one and tanking the other — it forces balance, which is why it beats accuracy on skewed classes. It's the wrong summary when the two errors have very different costs (use $F\beta$ to tilt, $\beta > 1$ for recall-heavy), when you care about ranking across all thresholds (use AUC), or when the negatives matter too (F1 ignores TN entirely — a fraud model and a random model can share an F1 while differing on specificity).

Q4. Explain ROC-AUC and PR-AUC and when each is appropriate.

ROC-AUC = area under TPR-vs-FPR = $P(\text{a random positive outranks a random negative})$; a threshold-free, prevalence-invariant discrimination score, chance = 0.5. PR-AUC (average precision) = area under precision-vs-recall; its no-skill baseline is the prevalence, not 0.5. Use ROC-AUC when classes are roughly balanced or both errors matter symmetrically, or for cross-dataset comparison. Use PR-AUC when positives are rare and are the class you care about, because precision puts FP against TP (not against the vast TN pool), so it registers false alarms that FPR — with the huge TN denominator — hides.

Q5. A model shows ROC-AUC 0.92 offline but the on-call team complains of endless false alarms in production. What happened?

Prevalence. ROC-AUC is invariant to base rate, so a model that discriminates well on a balanced offline sample keeps its 0.92 when deployed on a 1%-positive stream — but precision is not invariant: with 99% negatives, even a small FPR generates far more false positives than there are true positives, so the alerts the team sees are mostly wrong. Listing 2 measures exactly this drift (ROC-AUC 0.951 $\rightarrow$ 0.756 while PR-AUC 0.867 $\rightarrow$ 0.266). The fix is to evaluate with PR-AUC / precision-at-recall at the production prevalence, then set the threshold for a tolerable alert precision.

Q6. Is the 0.5 threshold optimal? How do you choose an operating point?

0.5 is a convention, not an optimum — it's optimal only under equal error costs and calibrated probabilities. The threshold is a business decision layered on the model: sweep the precision/recall curve on a *validation* set and pick the point that maximizes your objective — max F1, or the highest recall subject to precision $\geq$ target, or minimum expected cost given a cost matrix. Listing 3 shows the same model becoming three products: default 0.5 (P 0.974/R 0.664), max-F1 at 0.360 (P 0.893/R 0.749), precision $\geq$ 0.95 at 0.510. Choose on validation, report on test — and expect slight slippage because operating points don't transfer perfectly.

Q7. What does it mean for a classifier to be calibrated, and how is that different from being accurate?

Calibrated: among cases assigned probability p , a fraction p are actually positive — the numbers mean what they say. This is orthogonal to discrimination/accuracy: a model can rank every positive above every negative (AUC 1.0) yet output 0.51 vs 0.49, badly calibrated; or be well-calibrated but weak at ranking. You need calibration whenever a decision consumes the probability itself — expected-value thresholds, risk pricing, triage, stacking scores into another model. Diagnose with a reliability curve (predicted vs empirical frequency by bin); Listing 4's raw SVM traces an S-curve (pred 0.15 $\rightarrow$ actual 0.01).

Q8. Platt scaling vs isotonic regression — mechanism and when to use each.

Both are post-hoc maps from raw scores to calibrated probabilities, fit on a held-out set. Platt fits a one-parameter sigmoid (logistic regression on the scores) — cheap, low-variance, ideal when the miscalibration is sigmoidal or calibration data is scarce, but it can't fix non-sigmoidal distortions. Isotonic fits a free monotonic step function — corrects any monotone distortion, but is higher-variance and overfits small calibration sets. Rule of thumb: Platt for small data or sigmoidal error, isotonic when you have plenty of calibration data and an arbitrary monotone shape. Listing 4: both cut Brier $0.061 \rightarrow \sim 0.043$; with more data isotonic pulls ahead. Never calibrate on training data.

Q9. What is a proper scoring rule, and why does it matter?

A scoring rule for probabilistic predictions that is optimized (in expectation) by reporting your true beliefs — you cannot lower your expected score by lying about your confidence. Log loss and Brier are proper; accuracy is not (it only sees the hard label, so any probability > 0.5 scores the same). It matters because if you train or select on an improper rule you invite miscalibrated overconfidence, whereas a proper rule rewards being both sharp and honest. When the deliverable is a probability, the selection metric should be proper — otherwise you optimize for the wrong thing.

Q10. Compare log loss and Brier score.

Log loss = $-\text{mean log}(p \text{ at the true class})$: unbounded above, so it punishes confident-wrong catastrophically — a probability near 0 on an event that occurs contributes ~ 34.5 alone (Listing 5). Brier = mean squared error between probability and outcome: bounded $[0,1]$, gentler on a single disaster, and it decomposes into reliability – resolution + uncertainty, cleanly separating calibration from discrimination. Reach for log loss when one overconfident miss is unacceptable and you want maximum sensitivity; reach for Brier when you want a bounded, decomposable, outlier-tolerant number. Both are proper; both grade probabilities, not labels.

Q11. Two models have identical accuracy but very different log loss. How is that possible, and which ship?

Accuracy only checks which side of 0.5 each prediction falls; log loss reads the confidence. A model that's right at 0.51 and a model that's right at 0.95 share accuracy but the confident one has far lower log loss on its hits — and a model that's occasionally confident-and-wrong pays a huge log-loss penalty accuracy never sees (Listing 5: a bold model with *lower* accuracy 0.902 beats a timid always-0.6 model's log loss despite the timid one scoring 1.000 accuracy). Which ship depends on the deliverable: if you consume probabilities, prefer the better log loss; if you only act on the hard label at a fixed threshold, accuracy (at that threshold) is what matters.

Q12. RMSE vs MAE — how do they differ and how do you choose?

RMSE squares residuals before averaging (then roots), so large errors dominate; MAE averages absolute residuals, so every unit counts equally. Consequences: RMSE is outlier-sensitive (Listing 6: three big misses push RMSE $5.11 \rightarrow 6.75$ while MAE moves $4.08 \rightarrow 4.40$), $\text{RMSE} \geq \text{MAE}$ always, and they're minimized by different predictors — RMSE by the conditional mean, MAE by the conditional median. Choose RMSE when large errors are disproportionately costly (and it's the loss your squared-error model optimizes); choose MAE when errors should count linearly and you want robustness to a few bad rows.

Q13. When does MAPE break, and what replaces it?

$\text{MAPE} = \text{mean } |(y - \hat{y})/y|$ divides by the target, so it explodes as targets approach zero — Listing 6 sets five targets to 0.1 and MAPE jumps $8.5\% \rightarrow 497\%$ with unchanged predictions — and it's undefined at exactly zero. It's also asymmetric, penalizing over-prediction more than under-prediction, which biases models that optimize it. Replace it with sMAPE (symmetric, bounded) or WAPE / weighted MAPE (sum of absolute errors over sum of actuals, which dilutes the near-zero denominators). MAPE is only safe when targets are comfortably bounded away from zero and you want a scale-free percentage for cross-series comparison.

Q14. Can R^2 be negative? Why does adding features never decrease it, and what fixes that?

Yes — $R^2 = 1 - SS_res/SS_tot$ compares your model to the mean baseline, so any model worse than "predict the mean" is negative (Listing 6: predicting 0 everywhere scores -24.6). And on the training set R^2 never decreases when you add features, even pure noise, because extra parameters can only reduce in-sample residuals (Listing 6: +40 noise columns nudged R^2 $0.915 \rightarrow 0.919$). Adjusted R^2 penalizes for parameter count and so falls on junk features ($0.914 \rightarrow 0.912$); an honest alternative is out-of-sample R^2 on held-out data. Use adjusted or out-of-sample R^2 whenever comparing models with different numbers of features.

Q15. You need a single number for a search/recommendation system. Which metric and why?

A ranking metric, because the deliverable is an ordering and classification metrics ignore position. Pick by the product: MRR (mean reciprocal rank) when the user wants one good answer and only the first hit's rank matters (known-item search); MAP when relevance is binary and packing *all* relevants high matters; NDCG when relevance is graded (0–3) and/or you need cross-query comparability, since it discounts by log position and normalizes by the ideal ranking. Report it @k (NDCG@10) because users see a finite window. Listing 7 shows why order-blind metrics fail — recall@k over the whole list scores a top-loaded and bottom-loaded ranking identically.

Q16. Explain NDCG, including the discount and the normalization.

$DCG = \sum rel_i / \log_2(i+1)$: each item's (possibly graded) relevance is discounted by a logarithm of its rank, so a relevant item lower down contributes less — encoding that users pay more attention to the top. NDCG divides DCG by IDCG, the DCG of the ideal ordering (relevances sorted descending), giving a 0–1 score comparable across queries with different numbers of relevant items. Listing 7: best-first graded relevances score 1.000, worst-first 0.614. The two design choices are the position discount (why NDCG rewards ordering, not just presence) and the per-query normalization (why you can average it across heterogeneous queries).

Q17. Model B beats model A by 0.5% accuracy on the test set. Do you ship it?

Not on that number alone — 0.5% may be inside the noise. Test it: McNemar's test on the paired predictions (only the rows where they disagree carry signal), or a bootstrap/cross-validated confidence interval on the metric gap, and ship only if the interval clears zero. Listing 8 shows both outcomes: a real model change gives a CI [+0.039, +0.055] (clearly non-zero) while two models differing only by random seed give +0.0022 with CI [−0.0004, +0.0048] spanning zero — a mirage. A single split is one draw; report a spread, not a point.

Q18. Why McNemar's test rather than comparing two accuracy numbers, and why does it use only disagreements?

Because the two models are evaluated on the *same* test rows, so their errors are correlated — an unpaired comparison of two accuracies ignores that pairing and loses power. McNemar builds the 2×2 table of discordant pairs (A-right/B-wrong vs A-wrong/B-right) and tests whether that split is lopsided under the null of equal error rates. The concordant cells (both right, both wrong) carry no information about *which* model is better — they'd cancel — so only the discordant counts enter the statistic. Listing 8: 184 vs 54 discordant, χ^2 58.3, $p < 0.0001$.

Q19. How does a bootstrap confidence interval on a metric work, and what's its advantage?

Resample the test set with replacement to the same size, recompute the metric (or the gap between two models) on each resample, repeat a few thousand times, and take the 2.5/97.5 percentiles of the resulting distribution as a 95% CI. Its advantage is generality: it works for *any* metric — AUC, F1, NDCG — with no closed-form variance and no distributional assumption, and for model comparison you resample the same indices for both models (paired) to cancel the which-rows-landed-in-test variance. Listing 8 uses it on an AUC gap; the interval clearing zero is the ship/no-ship signal.

Q20. What is conformal prediction, and what exactly does it guarantee?

A wrapper that turns any trained model's scores into prediction *sets* with a finite-sample coverage guarantee. Split conformal: on a held-out calibration set compute nonconformity scores (e.g. $1 - \text{predicted prob of the true class}$), take the $[(n+1)(1-\alpha)]/n$ quantile, and at test time output every label whose nonconformity falls below it. Guarantee: the set contains the true label with probability $\geq 1-\alpha$, assuming only exchangeability — no distributional form, no large-sample asymptotics. Listing 9 hits 0.904 and 0.951 empirical coverage at 90%/95% targets. It's the one uncertainty method you can bolt onto any model for a coverage promise.

Q21. What's the price of conformal's guarantee, and how does calibration differ from it?

The price is variable set size: to keep coverage, the set expands where the model is unsure (two or more labels) and can be empty where nothing is plausible — the size is the honesty, an explicit "here are the possibilities at 95%." Calibration is a different promise: it makes the scalar probabilities accurate *on average* (70% of the 0.7-cases are positive) but gives no per-instance guarantee. Conformal gives a marginal coverage guarantee per prediction (a set that provably contains the truth $1-\alpha$ of the time) at the cost of set-valued output. Calibration → trustworthy numbers; conformal → a coverage contract.

Q22. Precision-recall or ROC for early-warning fraud detection at 0.5% prevalence — argue it.

Precision-recall. At 0.5% positives the negative pool is enormous, so ROC's FPR barely registers thousands of false positives while precision — FP against TP — feels every one, and the on-call team lives on precision (how many of my alerts are real) and recall (how much fraud did I catch). ROC-AUC would look reassuringly high and hide an unusable alert precision (Listing 2's mechanism). Report PR-AUC for the ranking, then fix an operating point by precision-at-recall against the fraud team's review capacity, and set the threshold there — the threshold, not AUC, is what ships.

Q23. Micro vs macro vs weighted averaging for multiclass metrics — what's the difference?

Macro averages the per-class metric with equal weight, so rare classes count as much as common ones — use it when minority-class performance matters (macro-F1 is the standard imbalanced-multiclass headline). Micro pools all TP/FP/FN across classes before computing, so it's dominated by frequent classes and, for single-label problems, equals overall accuracy. Weighted averages per-class metrics by class support — a compromise that respects prevalence while still being per-class. The interview point: on imbalanced multiclass, macro exposes the failure on rare classes that micro/accuracy conceal, so state which you're quoting and why.

Q24. How should cross-validation choice change for grouped, imbalanced, or time-series data?

Match the split to the dependence structure. Imbalanced → StratifiedKFold to keep class ratios stable across folds (plain KFold can hand a fold zero positives). Grouped (multiple rows per patient/user/session) → GroupKFold so no group straddles the train/test boundary, else the model memorizes the group and leaks. Time series → forward-chaining / TimeSeriesSplit (train on past, validate on future); a shuffled split lets the model peek at the future and manufactures optimism (Chapter 4's walk-forward vs shuffled gap). The default random KFold is only correct for i.i.d. data with balanced classes.

Q25. Should evaluation metrics ever be computed inside the cross-validation loop, or only at the end?

Compute the metric per fold and aggregate (mean $\pm$ spread), never pool predictions from a single split. Per-fold scoring gives you the variance — the honesty interval — that a single number hides, and it's what lets you tell a real gain from noise. Any step that *fits* — imputation, scaling, encoding, feature selection, resampling, threshold and calibration choice — must live inside the loop and be refit per fold, or its statistics leak the validation fold (Chapter 9's ledger). The metric itself doesn't leak, but everything feeding it can, so the whole preprocessing-plus-scoring stack goes inside the fold.

Q26. What is Cohen's kappa and when would you prefer it over accuracy?

Cohen's kappa = $(p_{\text{observed}} - p_{\text{chance}}) / (1 - p_{\text{chance}})$: agreement corrected for the agreement expected by chance given the class marginals. It's useful when chance agreement is high — heavy imbalance or when comparing to a majority-vote baseline — because it discounts the "free" accuracy a trivial classifier earns; kappa 0 means no better than chance, 1 means perfect. It also generalizes to inter-annotator agreement for label-quality audits, and weighted kappa penalizes ordinal misclassifications by distance. Prefer it (or balanced accuracy/MCC) over raw accuracy whenever the base rates make accuracy look good for free.

Q27. Why is Matthews correlation coefficient (MCC) considered a robust single-number classification metric?

MCC is the correlation between predicted and actual binary labels, computed from all four confusion cells: $(TP \cdot TN - FP \cdot FN) / \sqrt{((TP + FP)(TP + FN)(TN + FP)(TN + FN))}$, ranging -1 to $+1$. Because it uses TP, TN, FP, and FN symmetrically, it only scores high when the model does well on *both* classes — unlike F1, which ignores TN and can be inflated on imbalanced data. That balance makes it a strong default headline for imbalanced binary problems; a high MCC is hard to fake with a majority-class strategy. Its cost is lower interpretability than precision/recall, so pair it with the operating-point numbers stakeholders act on.

Q28. Walk through how you'd design the evaluation for a new binary classifier end to end.

Start from the decision and its cost asymmetry, which dictates the metric — rare valuable positives → PR-AUC plus precision-at-recall; balanced/both-errors → ROC-AUC or MCC; probabilities consumed downstream → add log loss/Brier and a reliability curve. Build an honest split matching the data's structure (stratified/group/time-series), keep every fitted step inside CV, and reserve a validation slice for the threshold and any calibration. Report the primary metric with a cross-validated spread and a bootstrap CI, choose the operating point on validation, calibrate if probabilities matter, and A/B or McNemar-test against the incumbent so you ship only when the interval clears zero. Optionally wrap conformal for a coverage guarantee. The through-line: the metric mirrors the decision, and every number carries an uncertainty band.